Сегментация изображений

В низкоуровневых методах обработки изображений на входе и выходе присутствуют изображения. В методах обработки изображений среднего и высокого уровня на вход подается непосредственно само изображение, а на выходе процесса обработки получаем некоторые свойства и атрибуты, извлеченные из изображения.

Сегментация делит изображение на составные части и объекты. Степень детализации этого деления зависит от решаемой задачи. Сегментация сложных изображений является весьма нетривиальной задачей обработки изображений.

Как правило, алгоритмы сегментации монохромных изображений основаны на одном из двух базовых свойств яркости изображения: разрывности и однородности. В первом случае подход состоит в разбиении изображения на части, исходя из резких перепадов значений яркости, которые происходят, например, на границах объектов. Вторая группа методов осуществляет разбиение изображений на области, однородные в смысле определенных, заранее заданных критериев.

Рассмотрим некоторые методы, основанные на этих двух подходах, применительно к монохромным изображениям. Начнем с представления методов, которые пригодны для обнаружения разрывов яркости, например, точек, линий, краев и перепадов. Выделение контуров объектов долгие годы являлось основой алгоритмов сегментации. Помимо обнаружения перепадов яркости как таковых, мы рассмотрим подход к обнаружению прямых участков, основанный на *преобразовании Хафа*. После обсуждения выделения контуров рассмотрим методы пороговых преобразований. Пороговая обработка также является одним из фундаментальных подходов к сегментации изображений, завоевавших популярность в приложениях, требующих быстрое принятие решения.

1. Обнаружение контуров (краев, границ)

В сфере обработки изображений, извлечение геометрических характеристик изображения очень распространенная проблема. На протяжении многих лет были разработаны различные подходы для извлечения этих характеристик. Эти различные подходы могут быть охарактеризованы и классифицированы по-разному.

Некоторые методы включают **глобальное изучение** (рассмотрение) изображений, в то время как другие привлекают только **локальное рассмотрение** каждого пикселя в изображении.

1.1. Что такое обнаружение контуров?

С точки зрения распознавания и анализа объектов на изображении наиболее информативными являются не значения яркостей объектов, а характеристики их границ – контуров. Другими словами, основная информация заключена не в яркости отдельных областей, а в их очертаниях. Задача выделения контуров состоит в построении изображения именно границ объектов и очертаний однородных областей.

Как правило, граница предмета на фотографии отображается перепадом яркости между двумя сравнительно однотонными областями. Но перепад яркости может быть вызван также текстурой предмета, тенями, бликами, перепадами освещенности, и т. п.

Будем называть *контуром изображения* совокупность его пикселей, в окрестности которых наблюдается скачкообразное изменение функции яркости. Так как при цифровой обработке изображение представлено как функция целочисленных аргументов, то контуры представляются линиями шириной, как минимум, в один пиксель. Если исходное изображение, кроме областей постоянной яркости, содержит участки с плавно меняющейся яркостью, то непрерывность контурных линий не гарантируется. С другой стороны, если на “кусочно-постоянном” изображении присутствует шум, то могут быть обнаружены “лишние” контуры в точках, которые не являются границами областей.

При разработке алгоритмов выделения контуров нужно учитывать указанные

особенности поведения контурных линий. Специальная дополнительная обработка выделенных контуров позволяет устранять разрывы и подавлять ложные контурные линии.

Процедура построения бинарного изображения границ объектов обычно складывается из двух последовательных операций: выделения контуров и их пороговой обработки.

Исходное изображение подвергается линейной или нелинейной обработке, с реакцией на перепады яркости. В результате этой операции формируется изображение, функция яркости которого существенно отличается от нуля только в областях резких изменений яркости изображения. Пороговой обработкой из этого изображения формируется контурный объект. Выбор порога на втором этапе должен производиться из следующих соображений. При слишком высоком пороге могут появиться разрывы контуров, а слабые перепады яркости не будут обнаружены. При слишком низком пороге из-за шумов и неоднородности областей могут появиться ложные контуры.

1.2. Обнаружение перепадов с помощью функции `edge` в MATLAB

Основная идея обнаружения перепадов базируется на поиске участков изображения, где яркость быстро меняется, с помощью следующих общих двух критериев:

1. найти участки, где первая производная яркости превосходит по модулю некоторый заранее заданный порог.
2. найти места, где вторые производные яркости имеют пересечения нулевого уровня.

Функция IPT `edge` реализует несколько оценок производных для использования в приведенных выше критериях. Для некоторых из этих оценок имеется возможность указать, к каким именно перепадам чувствительная данная оценка: к горизонтальным, вертикальным или к перепадам обоих типов. Общий синтаксис этой функции выглядит следующим образом:

```
[g, t] = edge(f, 'method', parameters),
```

где `f` – это входное изображение, `method` – одна из оценок, перечисленных в табл. 1, а `parameters` – дополнительные параметры, которые объясняются ниже. Выходом функции служит логический массив `g`, в котором стоят единицы там, где обнаружены точки перепада на `f`, и нули там, где перепады не обнаружены. Аргумент `t` является необязательным, в него записывается значение порога, которое функция `edge` использовала для сравнения с градиентами точек изображения.

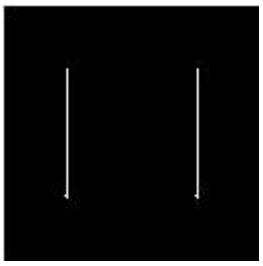
Табл. 1. Детекторы контуров, доступные в функции `edge`.

Детектор контуров	Основные свойства
Собела (`'sobel'`)	Обнаруживает контуры с помощью приближений Собела первых производных
Превитта (`'prewitt'`)	Обнаруживает контуры с помощью приближений Превитта первых производных
Робертса (`'roberts'`)	Обнаруживает контуры с помощью приближений Робертса первых производных
Лапласиан гауссиана (`'log'`)	Обнаруживает контуры, выполняя поиск пересечений нулевого уровня после фильтрации $f(x, y)$ гауссианом
Пересечения нулевого уровня (`'zerocross'`)	Обнаруживает контуры, выполняя поиск пересечений нулевого уровня после фильтрации $f(x, y)$ фильтром, заданным пользователем. Этот метод часто сочетается с фильтром Лапласианом гауссиана, но и с любым фильтром, который аппроксимирует вторую производную данных изображения. Если не указывать свой собственный фильтр, данный метод использует эквивалент следующего фильтра: `H = FSPECIAL('log', 13, 2);`

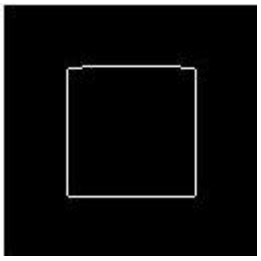
Канни (`'canny'`)	Обнаруживает контуры, выполняя поиск максимумов градиента $f(x, y)$. Градиент вычисляется от гауссиана. Метод использует два порога для нахождения сильных и слабых контуров. Слабые контуры включаются в выход, если они связаны с сильными. Следовательно, этот метод с большей вероятностью обнаруживает настоящие слабые контуры

Теперь давайте попробуем эти различные методы и параметры, чтобы увидеть разницу между ними:

```
T = 100;
f = zeros(128, 128);
f(32:96, 32:96) = 255;
[g1, t1] = edge(f, 'sobel', 'vertical');
imshow(g1);
t1
```



```
sigma = 1;
f = zeros(128,128);
f(32:96, 32:96) = 255;
[g3, t3] = edge(f, 'canny', [0.04 0.10], sigma);
figure, imshow(g3);
t3
```



2. Преобразование Хафа

В идеале рассмотренные выше методы должны обнаруживать только пиксели, принадлежащие краям и перепадам яркости. Однако па практике выделенные пиксели редко относятся только к этой категории в силу многих причин: воздействия шума, разрыва краев из-за неравномерного освещения и других факторов, которые вносят ложные перепады яркости в изображения. Поэтому за алгоритмом обнаружения краев обычно следует процедура компоновки выделенных пикселей краев в настоящие, осмысленные линии и краевые сегменты. Один из подходов к выполнению подобных действий основан на *преобразовании Хафа*.

Имея некоторое множество точек изображения (обычно двоичного), предположим, что требуется найти подмножества этих точек, которые лежат на прямых линиях. Один возможный подход заключается в построении всевозможных прямых, задаваемых парами этих точек, а потом в обнаружении точек, которые расположены близко к конкретным прямым. Проблема реализации такой процедуры связана с необходимостью рассмотрения $n(n-1)/2 \sim n^2$ прямых, а затем в выполнении порядка $n(n(n-1))/2 \sim n^3$ операций сравнения

всех точек с каждой из этих прямых. Вычислительная сложность такого решения позволяет применять его лишь в самых простых прикладных задачах.

Имея преобразование Хафа, возможен иной подход. Рассмотрим любую точку (x_i, y_i) и все проходящие через нее прямые. Все такие прямые удовлетворяют уравнению $y_i = ax_i + b$ для произвольных значений a и b. Однако если переписать это уравнение в виде $b = -ax_i + y_i$ и рассмотреть соответствующую ab-плоскость (называемую *пространством параметров* или *фазовым пространством*), то оно задаст единственную прямую для каждой фиксированной координатной пары (x_i, y_i). Более того, другой точке (x_j, y_j) соответствует своя прямая в пространстве параметров, и эти две прямые пересекаются в некоторой точке (a', b'), где a' – это угловой коэффициент, а b' – точка пересечения с осью y прямой, проходящей через точки (x_i, y_i) и (x_j, y_j) на xy-плоскости. На самом деле, у каждой точки этой прямой имеется прямая в пространстве параметров, причем все такие прямые пересекаются в точке (a', b'). Эти понятия иллюстрируются на рис. 1.

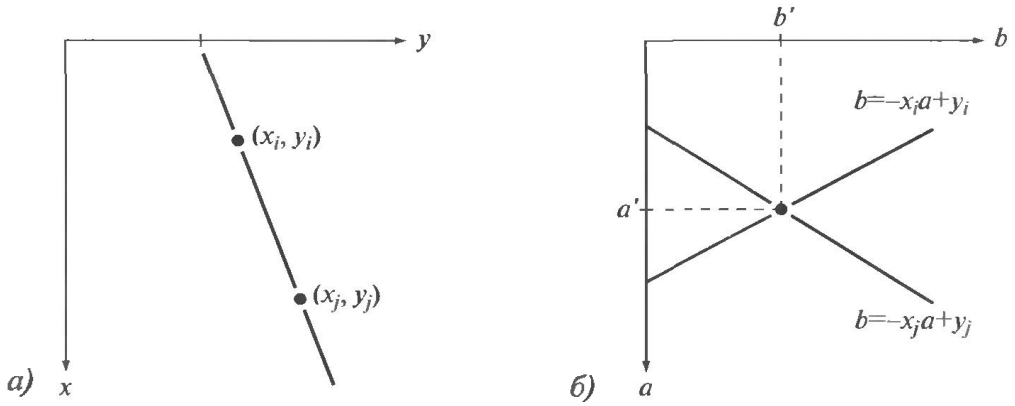


Рис. 1. *а*) Плоскость *xy; б*) Пространство параметров *ab*.

В принципе, можно построить на графике все параметрические прямые, соответствующие всем заданным точкам (x_i, y_i) изображения, а затем все линии изображения можно идентифицировать с помощью точек пересечения параметрических прямых. При этом возникает вычислительная трудность, связанная с тем, что число a (угловой коэффициент) стремится к бесконечности, когда прямая близка к вертикали. Один из способов справиться с этой проблемой состоит в представлении уравнения прямой с помощью ее вектора нормали:

$$x \cos\theta + y \sin\theta = \rho.$$

На рис. 2, *а*) приведена геометрическая интерпретация параметров ρ и θ. Горизонтальная прямая имеет $\theta = 0°$, а параметр ρ равен (положительной) координате пересечения прямой с осью x. Аналогично, вертикальная прямая имеет $\theta = 90°$, а ρ равно координате пересечения с положительной полуосью y или $\theta = -90°$, а ρ – координате пересечения с отрицательной полуосью y. Каждая синусоидальная кривая на рис. 2, *б*) отвечает семейству прямых линий, проходящих через некоторую точку (x_i, y_i). Точка пересечения кривых (ρ', θ') соответствует прямой, проходящей через обе точки (x_i, y_i) и (x_j, y_j).

Привлекательность преобразования Хафа с точки зрения вычислений происходит из возможности разбиения пространства параметров $\rho\theta$ на так называемые ячейки накопления, как показано на рис. 2, *в*), где $(\rho_{\min}, \rho_{\max})$ и $(\theta_{\min}, \theta_{\max})$ – предполагаемые диапазоны значений параметров. Обычно эти значения варьируются в интервалах $-90° < \theta < 90°$ и $-D < \rho < D$, где D – это расстояние между угловыми точками изображения. В ячейке с координатами (i, j) накапливается значение $A(i, j)$ для квадрата в пространстве параметров, соответствующего точке (ρ_i, θ_i). Вначале значения всех ячеек накопления равны нулю. Затем для каждой точки (x_k, y_k) выбранного множества на изображении полагаем параметр θ равным каждому разрешенному дискретному значению θ_j в ячейках θ-оси и находим соответствующее ему значение, решая уравнение $\rho = x_k \cos\theta + y_k \sin\theta$. Затем найденные величины ρ округляются

до ближайшего разрешенного дискретного значения ρ_i из ячеек ρ-оси. После этого значение соответствующей ячейки накопления увеличивается на единицу: $A(i, j) := A(i, j) + 1$. В конце процедуры ячейка $A(i, j)$ равна числу Q, которое означает, что Q точек плоскости xy лежат на прямой $x \cos \theta_j + y \sin \theta_j = \rho_i$. Точность попадания этих точек на прямую зависит от числа ячеек накопления в пространстве $\rho\theta$.

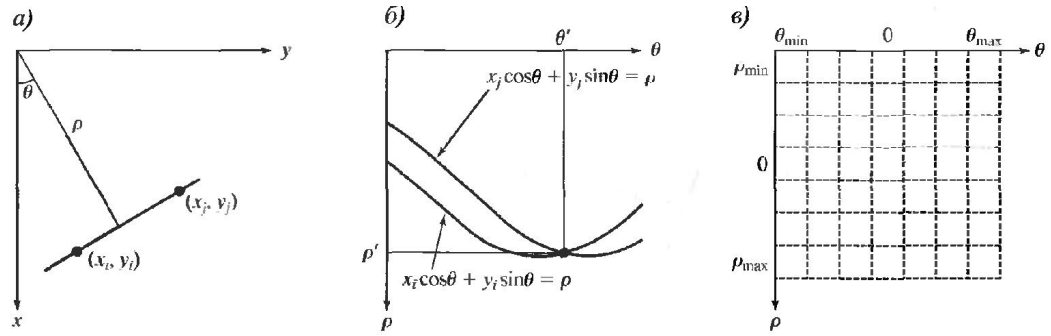


Рис. 2. *а*) Параметризация (ρ, θ) прямых на плоскости xy; *б*) Синусоиды в плоскости $\rho\theta$. Точка пересечения (ρ', θ') соответствует параметрам прямой, соединяющей точки (x_i, y_i) и (x_j, y_j); *в*) Разделение плоскости $\rho\theta$ на ячейки накопления.

Далее рассматривается М-функция для вычисления преобразования Хафа. Эта функция использует операции с *разреженными* матрицами, которые имеют малое число ненулевых элементов. Свойство разреженности позволяет получить выигрыш, как при хранении данных, так и при вычислениях. Имея матрицу A, преобразуем ее в разреженный матричный формат с помощью функции `sparse`, которая имеет синтаксис:

```
S = sparse(A).
```

Например,

```
>> A = [ 0 0 0 5
         0 2 0 0
         1 3 0 0
         0 0 4 0 ];
>> S = sparse (A)
S =
   (3,1)   1
   (2,2)   2
   (3,2)   3
   (4,3)   4
   (1,4)   5
```

Здесь перечислены все ненулевые элементы `S` вместе с их индексами строк и столбцов. Элементы упорядочены по столбцам.

Имеется форма обращения к функции `sparse` с пятью аргументами, которая используется чаще всего:

```
S = sparse(r, c, s, m, n).
```

Здесь `r` и `c` – это векторы индексов строк и столбцов ненулевых элементов матрицы, которую мы хотим перевести в разреженный формат. Параметр `s` представляет собой вектор значений, отвечающих индексным парам `(r, c)`, а `m` и `n` обозначают число строк и число столбцов построенной матрицы. Например, матрицу `S` из предыдущего примера можно прямо построить командой

```
>> S = sparse([3 2 3 4 1], [1 2 2 3 4], [1 2 3 4 5] , 4, 4);
```

Существуют и другие синтаксические формы обращения к функции `sparse`, которые

можно узнать из справочной информации по этой функции.

Имея разреженную матрицу S, порожденную любой формой функции sparse, можно вернуться к стандартному матричному представлению, применяя функцию full, синтаксис которой имеет вид:

```
A = full(S).
```

Пример. *Иллюстрация преобразования Хафа.*

Здесь мы продемонстрируем использование функции hough при обработке простого двоичного изображения. Прежде всего, построим изображение, у которого имеется несколько изолированных точек переднего плана.

```
>> f = zeros(101, 101);
>> f(1, 1)     = 1;   f(101, 1) = 1;   f(1, 101) = 1;
>> f(101, 101) = 1;   f(51, 51) = 1;
```

Рис. 3, *а*) показывает наше тестовое изображение. Теперь мы выполним преобразование Хафа и отобразим результат на графике.

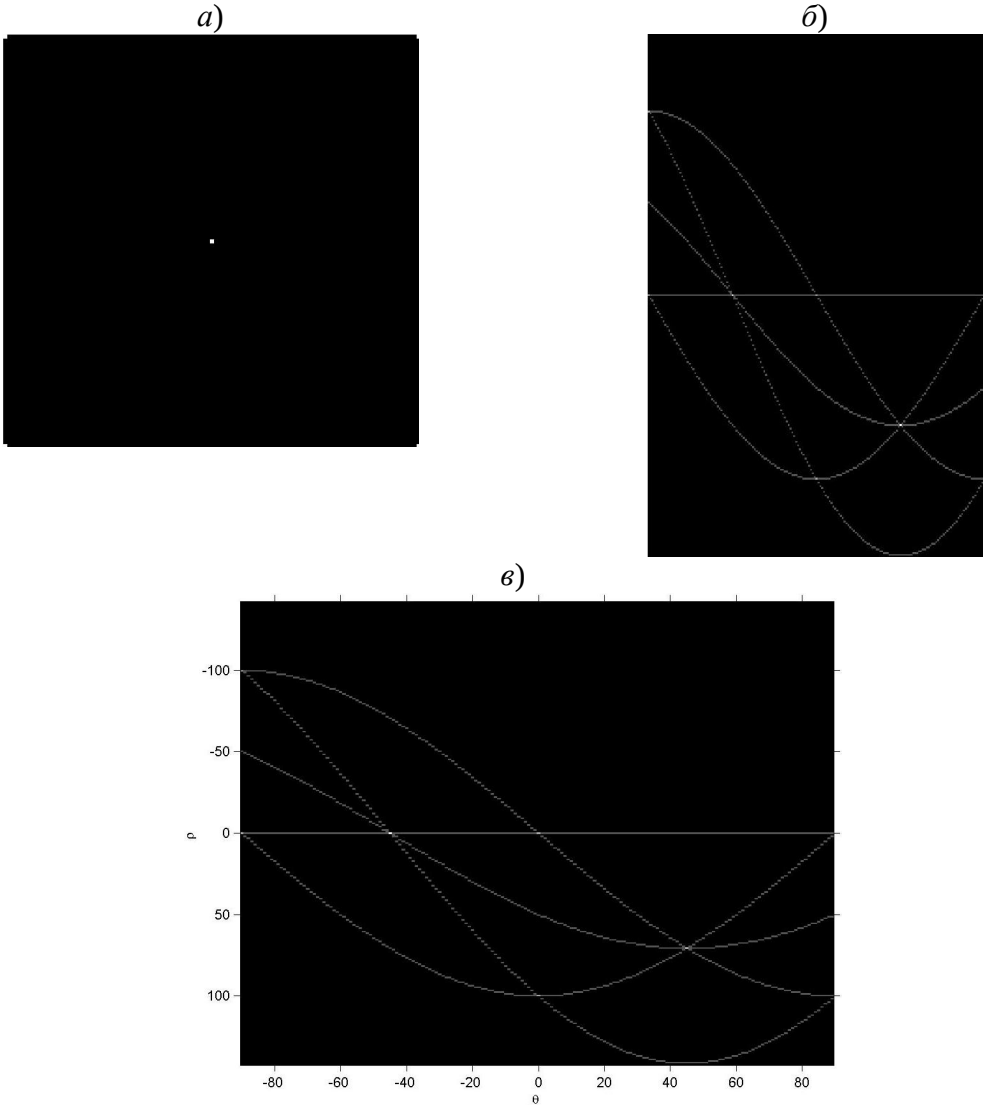


Рис. 3. ***а*) Двоичное изображение с пятью точками (четыре точки находятся в углах изображения); *б*) Преобразование Хафа, показанное с помощью функции imshow; в) Альтернативный график с размеченным осями. (Точки на рис. *а*) были увеличены для лучшей видимости).**

```
>> H = hough(f);
>> imshow(H, [ ])
```

На рис. 3, *б*) приведен результат, отображенный на графике стандартной функцией `imshow`. Однако более наглядная визуализация преобразования Хафа получается на большем графике, на котором по осям отмечены координаты. В следующем программном фрагменте функция `hough` вызывается с тремя аргументами. Два вторых выходных аргумента содержат величины θ и ρ, отвечающие каждой соответствующей паре индексов строки и столбца матрицы преобразования Хафа. Эти векторы `theta` и `rho` можно передавать функции `imshow` в виде дополнительных аргументов для разметки горизонтальных и вертикальных осей на графике. Функции `imshow` также передается опция `'notruesize'`. Команда `axis` подается для включения отображения осей и представления графика в прямоугольной форме. Наконец, функции `xlabel` и `ylabel` используются для помещения возле координатных осей греческих букв, представленных в стиле LATEX.

```
>> [H, theta, rho] = hough(f);
>> imshow(theta, rho, H, [ ], 'notruesize')
>> axis on, axis normal
>> xlabel('\theta') , ylabel('\rho')
```

На рис. 3, *в*) показан результирующий график с соответствующей разметкой осей. Пересечение трех синусоид в точках ±45° указывает на то, что на исходном изображении `f` имеется два множества, состоящие из трех коллинеарных точек. Пересечение двух синусоид в точках (θ, ρ) = (–90, 0), (–90, –100), (0, 0) и (0, 100) свидетельствует о присутствии четырех множеств коллинеарных точек, которые лежат вдоль вертикальных и горизонтальных осей.

2.1. Нахождение максимумов преобразования Хафа

Первый шаг использования преобразования Хафа для обнаружения линий и связывания состоит в нахождении локальных максимумов преобразования. Поиск множества максимумов преобразованием Хафа может быть вполне перспективным. В силу квантования пространства цифрового изображения и пространства параметров, а также по причине того, что края и перепады на типичных изображениях не являются совершенно прямыми, максимумы преобразования Хафа могут достигаться более чем в одной ячейке накопления. Эту сложность можно преодолеть с помощью следующей стратегии.

1. Найти ячейку преобразования Хафа, в которой лежит наибольшая величина, и записать ее местоположение.
2. Опорожнить (обнулить) ячейки в ближайшей окрестности положения, найденного на шаге 1.
3. Повторять шаги 1 и 2 до тех пор, пока желаемое число максимумов не будет найдено, или после достижения заданного порога.

Функция `houghpeaks` реализует эту стратегию.

2.2. Преобразование Хафа при обнаружении линий и связывании

После обнаружения множества локальных максимумов преобразования Хафа остается определить, есть ли сегменты краев, проходящие по соответствующим линиям, а также где они начинаются и заканчиваются. Для каждого максимума сначала следует найти положения всех ненулевых пикселей изображения, которые лежат на соответствующих прямых. Для этих целей написана функция `houghpixels`.

Пиксели, обнаруженные функцией `houghpixels`, необходимо сгруппировать в сегменты. Для этого можно воспользоваться следующей стратегией:

1. Повернуть пиксели на угол $90° - \theta$ так, чтобы они легли примерно вдоль вертикальной прямой.

2. Упорядочить пиксели в порядке возрастания величин их повернутых *x*-координат.
3. Использовать функцию `diff` для определения зазоров (щелей). Заполнить малые зазоры. Это дает эффект слияния примыкающих сегментов линий, которые разделены малыми промежутками.
4. Возвратить информацию о сегментах линий, которые длиннее некоторой минимальной пороговой длины.

Пример. *Использование преобразования Хафа для обнаружения линий и связывания.*

В данном примере вы воспользуемся функциями `hough`, `houghpeaks` и `houghlines` для обнаружения сегментов линий на двоичном изображении `f`, приведенном на рис. 4, *а*). Прежде всего, мы вычислим и покажем преобразование Хафа, используя меньший шаг угловой дискретизации, чем принятый по умолчанию ($\Delta\theta = 0.5$ вместо 1.0).

```
>> [H, theta, rho] = hough(f, 0.5);
>> imshow(theta, rho, H, [ ], 'notruesize'), axis on, axis normal
>> xlabel('\theta') , ylabel('\rho')
```

Затем с помощью функции `houghpeaks` мы находим 5 локальных максимумов преобразования Хафа, которые, скорее всего, имеют существенный смысл.

```
>> [r, c] = houghpeaks(H, 5);
>> hold on
>> plot(theta(c), rho(r), 'linestyle', 'none', 'marker', 's', 'color', 'w')
```

На рис. 4, *б*) изображено преобразование Хафа с указанием положения найденных максимумов. Наконец, применяем функцию `houghlines` для нахождения и связывания сегментов линий, а найденные сегменты помещаем на исходное двоичное изображение с помощью функций `imshow`, `hold on` и `plot`:

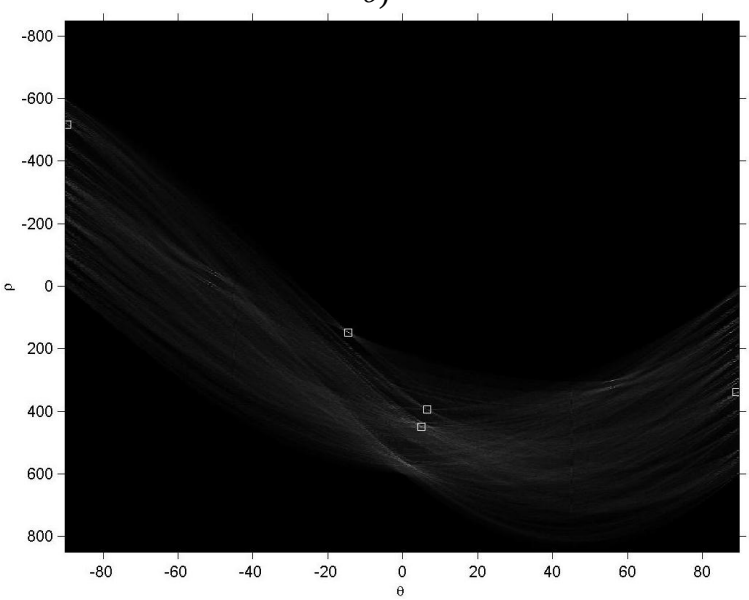
```
>>  lines = houghlines(f, theta, rho, r, c)
>>  figure, imshow(f), hold on
>>  for k = 1 :length(lines)
   xy = [lines(k).point1 ; lines(k).point2];
   plot(xy(:,2), xy(:,1), 'LineWidth', 4, 'Color', [.6 .6 .6]);
   end
```

На рис: 4, *в*) показано результирующее изображение с обнаруженными сегментами, которые обозначены жирными серыми линиями.

а)



б)



в)

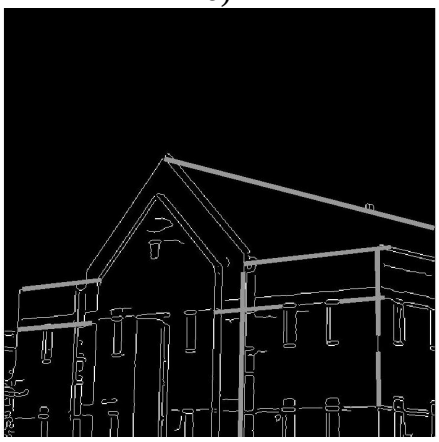


Рис. 4. *а*) Исходное изображение; *б*) Преобразование Хафа с пятью локальными максимумами; *в*) Сегменты линий, соответствующие максимумам преобразования Хафа.

3. Пороговая обработка

Методы пороговой обработки занимают центральное место в прикладных задачах сегментации изображений благодаря интуитивно понятным свойствам и простоте реализации. В этом пункте рассмотрим методы автоматического выбора пороговых величин, а также способы изменения порогов в соответствии с локальными свойствами изображений.

Предположим, что представленная ниже гистограмма соответствует некоторому изображению $f(x, y)$, которое содержит светлые объекты на темном фоне, причем яркости всех пикселей сосредоточены вблизи двух преобладающих значений. Очевидный способ выделения объектов из окружающего фона состоит в выборе порога T, который разделяет эти две моды яркости. Тогда любая точка (x, y), для которой $f(x, y) \geq T$, называется *точкой объекта*, а в противном случае она называется *точкой фона*.

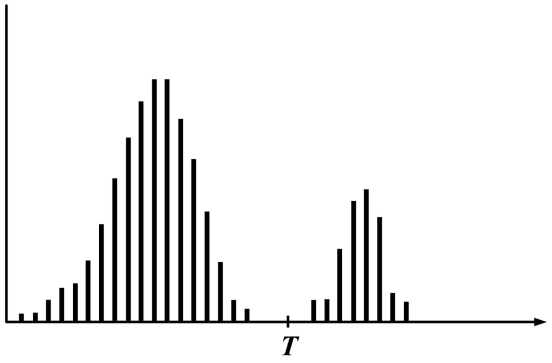


Рис. 1 Выбор порога на основании визуального анализа бимодальной гистограммы.

Иными словами, изображение $g(x, y)$, которое получается в результате этого порогового преобразования, определяется следующим образом:

$$g(x,y) = \begin{cases} 1 & \text{при } f(x,y) \geq T, \\ 0 & \text{при } f(x,y) < T. \end{cases}$$

Таким образом, пиксели со значениями 1 отвечают объектам, а пиксели, которым присвоено значение 0, соответствуют фону исходного изображения. Если T является константой (т. е. не зависит от координат (x, y)), то такой порог называется *глобальным*. Методы выбора глобальных порогов обсуждаются в пункте 3.1. В 3.2 рассматриваются способы вариации порогов, которые в этом случае называются *локальными* или

адаптивными.

3.1. Обработка с глобальным порогом

Один способ выбора порога заключается в визуальном изучении гистограммы изображения. Гистограмма на рис. 1 имеет две отчетливые моды, поэтому легко выбрать разделяющий их порог. Другой подход к выбору T основан на методе проб и ошибок, когда выбираются различные пороги и проверяются до тех пор, пока результат пороговой обработки не станет удовлетворять наблюдателя. Такой метод особенно эффективен при наличии хорошей интерактивной среды исследования, которая позволяет пользователю «двигать» параметры обработки в ручном графическом режиме и наблюдать немедленный результат этих изменений.

Для автоматического выбора порога предложена следующая интерактивная процедура:

1. Выбрать некоторую начальную оценку для значения порога T. (Предлагаемая величина равна среднему значению между максимумом и минимумом яркости изображения).
2. Совершить сегментацию с помощью порога T. В результате образуются две группы пикселей: G_1 и G_2 Область G_1 состоит из пикселей, яркость которых больше или равно T, а яркость пикселей из G_1 меньше T.
3. Вычислить среднюю яркость пикселей μ_1 и μ_2 по областям G_1 и G_2.
4. Вычислить новое значение порога:

$$T = \frac{1}{2}\left(\mu_1 + \mu_2\right).$$

5. Повторять шаги с 2-го по 4-ый до тех пор, пока разность порогов T соседних итераций не станет меньше наперед заданного значения T_o.

В пакете IPT имеется функция `graythresh`, которая вычисляет пороги по методу Отсу. Функция `graythresh` принимает входное изображение, вычисляет его гистограмму, а затем находит пороговую величину. Функция возвращает порог в нормализованном виде из интервала [0, 1], поэтому его следует правильно масштабировать перед использованием. Форма вызова функции имеет вид

```
T = graythresh(f),
```

где `f` – это входное изображение, а `T` – результирующий порог. Поскольку величина порога отнормирована к диапазону от 0 до 1, ее следует преобразовать перед использованием. Например, если `f` – это изображение класса `uint8`, то нужно сначала умножить T на 255, а потом применять для пороговой обработки.

Пример. *Вычисление глобальных порогов.*

В этом примере иллюстрируется описанная выше итеративная процедура, а также метод Отсу применительно к полутоновому изображению `f` отсканированного текста. Итеративный метод реализуется следующим способом:

```
T = 0.5*(double(min(f(:)))+double(max(f(:))));
done = false;
while ~done
   g = f >= T;
   Tnext = 0.5*(mean(f(g))+mean(f(~g)));
   done = abs (T-Tnext) < 0.5;
   T = Tnext;
end
```

Для этого конкретного изображения цикл `while` совершается четыре раза и прерывается, когда T равно 101.47.

Теперь вычислим порог с использованием функции `graythresh`:

```
>> T2 = graythresh(f)
T2 =
     0.3961
>> T2 * 255
ans =
     101
```

Результаты обработки этими двумя порогами практически неотличимы друг от друга.

3.2. Обработка с адаптивным порогом

Глобальная пороговая обработка может не дать желаемого результата, если фон изображения сильно неоднороден по яркости. В таких случаях необходимо применять предварительную обработку для компенсации перепадов фоновой яркости, после него можно совершать глобальное пороговое преобразование. Пороговое преобразование с переменной пороговой функцией $T(x, y)$:

$$g(x,y)=\begin{cases}1 & \text{при } f(x,y)\geq T(x,y),\\ 0 & \text{при } f(x,y)<T(x,y),\end{cases}$$

где

$$T(x,y)=f_o(x,y)+T_o.$$

Изображение $f_o(x, y)$ представляет собой морфологическое размыкание f, а константа T_o равна результату применения функции `graythresh` к изображению f_o.

4. Упражнения

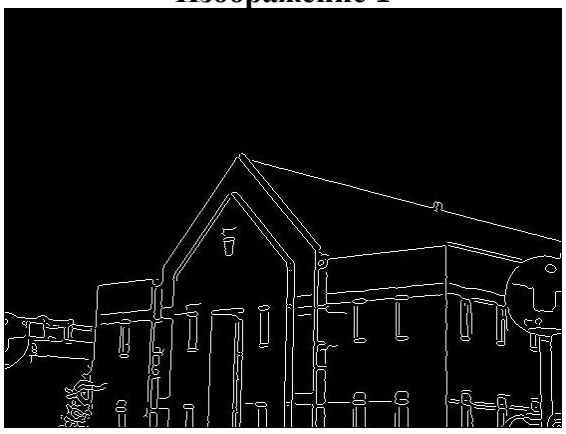
Часть 1. Контурная карта

1. Загрузите изображение `"building.jpg"` и сохраните в рабочем каталоге MATLAB.

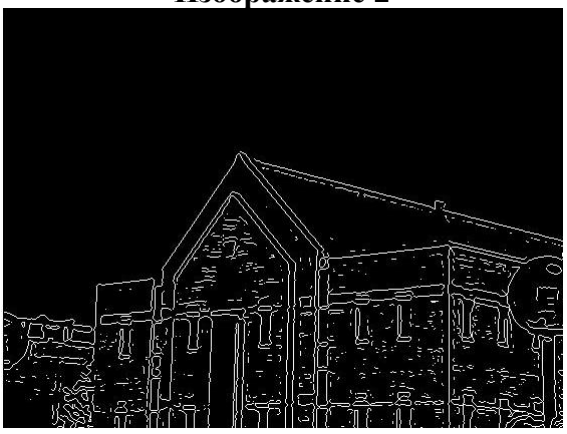


2. Определите которое изображение из следующих ниже является результатом применения детекторов «Пересечения нулевого уровня» (zerocrossing) и «Канни» (canny).

Изображение 1



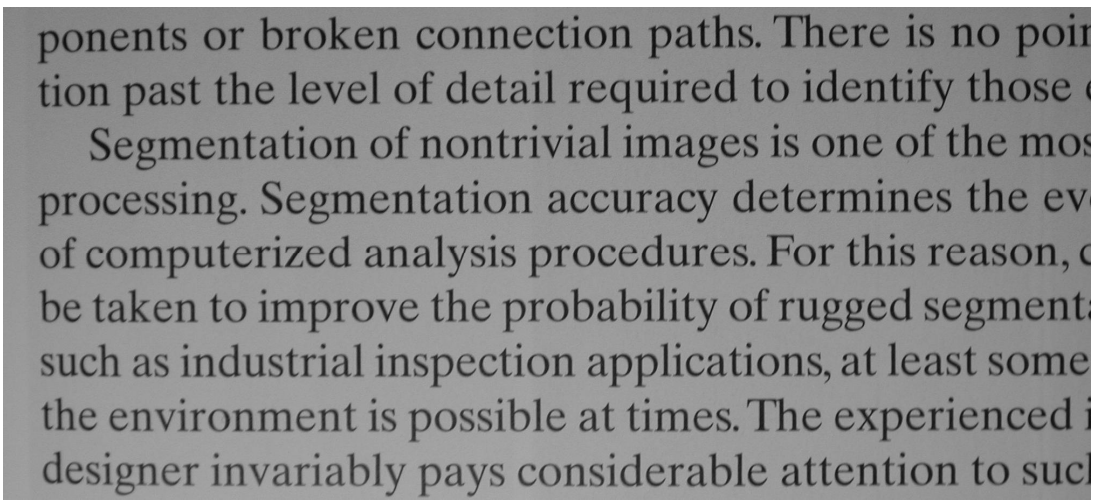
Изображение 2



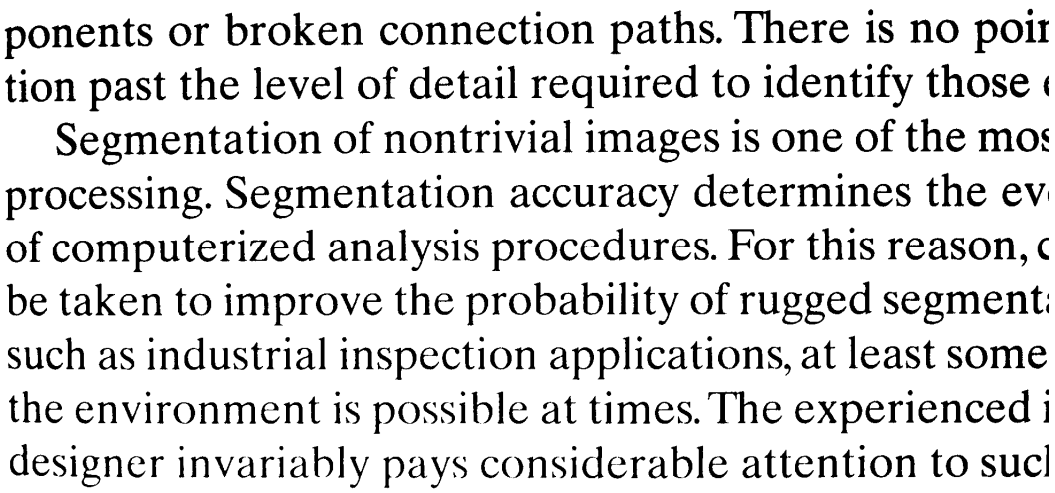
3. Воспроизведите результаты для исходного изображения `"building.jpg"`. В качестве рекомендаций попробуйте использовать различные значения `T` и `Sigma` для получения наилучших результатов.

Часть 2. Автоматическая пороговая обработка

1. Загрузите изображение `"text.jpg"` и сохраните в рабочем каталоге MATLAB.



2. Воспроизведите результат для получения изображения подобно следующему `"enhance_text.gif"`:


ponents or broken connection paths. There is no poir
tion past the level of detail required to identify those
Segmentation of nontrivial images is one of the mos
processing. Segmentation accuracy determines the ev
of computerized analysis procedures. For this reason,
be taken to improve the probability of rugged segment
such as industrial inspection applications, at least some
the environment is possible at times. The experienced i
designer invariably pays considerable attention to such

Часть 3. Преобразование Хафа

1. Выполните `hough` преобразование над результатом детектора Канни из Части 1.

2. Найдите достаточное количество максимумов в результатах преобразования Хафа, используя `houghpeaks`. Вы можете точно настроить ваши результаты, регулируя порог (`threshold`) и параметр влияния окрестности (neighborhood suppression – `nhood`).

3. Используйте эти максимумы с `houghlines`, как описано в справке по этой функции, для вывода сегментов линий исходного изображения. Пример на странице помощи поворачивает изображение, прежде чем находит контуры.

Конечный результат

- ✓ Часть 1
  - o сценарий под названием `edge_maps.m` (с комментариями к коду), который:
    - ▪ определяет, какой метод по обнаружению контуров был использован для двух изображений;
    - ▪ производит контурные карты с использованием детекторов «Пересечения нулевого уровня» (zerocrossing) и «Канни» (canny).
- ✓ Часть 2
  - o сценарий под названием `my_threshold.m`, который:
    - ▪ находит и выводит оптимальное значение `T` для `text.jpg`;
    - ▪ использует `T` для пороговой обработки `text.jpg` и отображает результат.
- ✓ Часть 3
  - o сценарий под названием `my_hough.m`, который использует функции преобразования Хафа для вывода сегментов линий на контурной карте детектора Канни изображения из Части 1.

5. Литература

1. Гонсалес Р. Цифровая обработка изображений / Р. Гонсалес, Р. Вудс. – М.: Техносфера, 2005. – 1072 с. **(стр. 812-874)**.

2. Гонсалес Р. Цифровая обработка изображений в среде MATLAB / Гонсалес Р., Вудс Р., Эддинс С.; пер. с англ. В.В. Чепыжева. – М.: Техносфера, 2006. – 616 с. **(стр. 396-424)**.